

Solución tarea 12

1. Generamos los imports necesarios para ejecutar el programa

- json y requests para las llamadas a la Api de Ollama
- gradio para utilizar la ventana gráfica

```
import json
import requests
import gradio as gr
```

2. Base de datos ficticia

Diccionario con varios lugares "destino" y su precio

```
# Base de datos inventada
ticket_prices = {
    "londres": "$799",
    "paris": "$899",
    "tokyo": "$1400",
    "berlin": "$499"
}
```

3. Obtenemos el precio del vuelo

Buscamos en el diccionario el destino (destination_city) y obtenemos el precio de este.

```
def get_ticket_price(destination_city):
    """
    Busca el precio del billete en el diccionario
    """
    print(f"DEBUG: Buscando precio para {destination_city}")
    city = destination_city.lower()
    return ticket_prices.get(city, "Precio desconocido para esta ciudad")
```

4. Definimos la herramienta (Esquema para Ollama)

Parte fundamental para que Ollama detecte que pueda utilizar una herramienta (el diccionario) si no tiene conocimientos para devolver la respuesta.

```
# Definición de la herramienta
tool_definition = {
    "type": "function",
    "function": {
        "name": "get_ticket_price",
        "description": "Get the price of a return ticket to the destination city.",
        "parameters": {
            "type": "object",
            "properties": {
                "destination_city": {
                    "type": "string",
                    "description": "The city that the customer wants to travel to",
                },
            },
            "required": ["destination_city"],
        },
    },
}
```

5. Llamadas a Ollama

5.1 Claves

```
OLLAMA_API_URL = "http://localhost:11434/api/chat"

#Modelo 3.1 por que es el que puede acceder a herramientas
OLLAMA_MODEL = "llama3.1"
```

5.2 Prompt del sistema

```
#Preparación de mensajes
messages = []

#Prompt del Sistema: Define el comportamiento y las herramientas disponibles
messages.append({
    "role": "system",
    "content": "Eres un asistente de viajes útil. Usa la herramienta get_ticket_price cuando te pregunten por precios."
})
```

5.3 Guardamos el historial para mostrarlo

```
#Guardamos el historial de la conversación
messages.extend(history)
```

5.4 Primera llamada a Ollama

En esta llamada le enviamos la petición del cliente y aquí Ollama tiene que decidir si responde de sus conocimientos o utiliza la herramienta proporcionada anteriormente

```
# PRIMERA LLAMADA A OLLAMA (Detectar Intención)
# Enviamos el historial junto con la definición de herramientas disponibles
payload = {
    "model": OLLAMA_MODEL,
    "messages": messages,
    "tools": [tool_definition], # Le decimos a Ollama qué funciones puede usar
    "stream": False,
    "options": {"temperature": 0}
}
```

5.5 Respuesta de ollama

```
try:
    response = requests.post(OLLAMA_API_URL, json=payload)
    response.raise_for_status()
    response_data = response.json()

    message_response = response_data.get("message", {})
    tool_calls = message_response.get("tool_calls", [])
```

5.5.1 Si el modelo decide no utilizar la herramienta

```
#Si el modelo responde sin usar herramientas
if not tool_calls:
    return message_response.get("content", "No entendí la pregunta.")
```

5.5.2 Si el modelo decide utilizar la herramienta

```
#Si el modelo decide usar una herramienta
# Primero, añadimos la respuesta "intención" del asistente al historial
messages.append(message_response)
```

```
#EJECUCIÓN DE HERRAMIENTAS
for tool in tool_calls:
    function_name = tool["function"]["name"]

    #Limpieza y parseo de argumentos
    args = tool["function"]["arguments"]
    if isinstance(args, str):
        try: args = json.loads(args)
        except: args = {}
```

```

#Verificamos qué función se solicitó
if function_name == "get_ticket_price":
    city = args.get("destination_city")
    #Ejecutamos nuestra función Python local
    price_result = get_ticket_price(city)

    #Preparamos el mensaje de resultado para el modelo
    tool_msg = {
        "role": "tool",
        "content": json.dumps({"price": price_result}),
        "name": function_name
    }
    #Id de llamada (requerido por estándares modernos)
    if "id" in tool: tool_msg["tool_call_id"] = tool["id"]

    messages.append(tool_msg)

```

Si utiliza la herramienta volvemos a llamar a Ollama para que genere una respuesta con los datos obtenidos

```

#SEGUNDA LLAMADA A OLLAMA (Respuesta Final)
# Enviamos el historial actualizado (incluyendo el resultado de la herramienta)
# para que el modelo genere una respuesta natural al usuario
final_payload = {
    "model": OLLAMA_MODEL,
    "messages": messages,
    "stream": False
}

final_response = requests.post(OLLAMA_API_URL, json=final_payload)
final_data = final_response.json()

#Devolvemos el texto final generado
return final_data["message"]["content"]

```

6. Interfaz con Gradio

Función que se activa al hacer “enter” en la ventana, hace la llamada al chat de Ollama y devuelve al gradio la respuesta.

```
def gradio_wrapper(message, history):
    #Añadir mensaje del usuario al historial
    history.append({"role": "user", "content": message})

    #Obtener respuesta del bot (pasamos todo el historial)
    bot_response = chat_logic(history)

    #Añadir respuesta del bot al historial
    history.append({"role": "assistant", "content": bot_response})

    #Devolver input vacío y el historial actualizado
    return "", history
```

```
with gr.Blocks(theme=gr.themes.Soft()) as demo:
    gr.Markdown("## ✈ Asistente de Viajes FlightAI (Con Ollama)")

    # type="messages" eliminamos el argumento explícito ya que Gradio 6.0 lo asume/usa por defecto
    chatbot = gr.Chatbot(label="Chat", height=400)
    msg = gr.Textbox(
        show_label=False,
        placeholder="Pregunta por precios (ej: Cuánto cuesta ir a Berlin?)"
    )
    clear = gr.ClearButton([msg, chatbot])

    msg.submit(gradio_wrapper, inputs=[msg, chatbot], outputs=[msg, chatbot])

if __name__ == "__main__":
    demo.launch()
```

La interfaz gráfica queda así:

